

# Transformers

## 1. from [8]

### 1. Attention

#### 1. Soft attention mechanisms use learned weights [8]:

##### 1. Channel-wise attention: SE Net. Related: see Squeeze Excitation Block in [Image Recognition](#).

1. Weights different channels by multiplying with channel-specific weights
2. winning entry of ImageNet classification task (2017)

##### 2. Spatial attention: Spatial Transformer Re-maps pixels and crops images.

1. Transforms each channel of spatial pixel in the same way.
2. Learns a 6 dof affine transformation in 2D.

#### 2. Modular way of incorporating Attention:

1. STN works in a similar way to SE Net.
2. Produces an output block of the same size as input.
  1. Insert between any pair of convolution layers (similar to SE Net)
  2. No change to existing architecture other than attention

#### 3. What a Spatial Attention can do

1. Can rotate, translate, reflect, or scale images
2. Transformation is image-specific
3. Magnification and translation automatically leads to cropping and attention!
  1. Attention is a special case of this more general approach
  2. Always learned spatial invariance and is therefore more discriminative than max-pooling

#### 4. Components of STN:

##### 1. Localization network

1. A regression network with 6 outputs, six image-specific values that are reshaped into a  $3 \times 2$  matrix  $\Rightarrow$  Defines the mapping between old and new image coordinates.
2. Grid generator: Transforms the 2D pixel coordinates ( $28 \times 28$  of them) with the generated matrix to map to (possibly fractional) coordinates.
3. Sampler: Creates new  $28 \times 28$  pixel values based on mapping.
4. Related: see Bilinear Interpolation in [BilinearInterpolation](#).
5. Six entries of  $A$  produced by localization network in image-specific manner.
6. Localization network can be CNN or even FCN.

##### 2. Grid generator

1. We need pixel values for every coordinate pair  $i, j \in 28 \times 28$  in transformed image.
2. Easier to use  $A$  to reverse map target coordinates  $i, j$  to coordinates  $x_{ij}, y_{ij}$  in original image.
3. Scaling factors inverses of eigenvalues of  $S$  in  $A = US$ .

##### 3. Interpreting generated coordinates

1. The pixel value  $i, j$  in the target image is equal to the pixel value  $x_{ij}, y_{ij} = [i, j, 1]A$  in source image.
2. Solution: Use interpolation from four surrounding pixels  $x_{ij}, y_{ij}$  in source image.
  1. Feature: results in (sub)-differential mapping, which is good for backpropagation.

## 2. from [2], demonstrates a special case of self-attention:

1. Self-attention is a seq2seq operation: a sequence of vectors goes in, and a sequence of vectors comes out. Let's call the input vectors  $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_t$  and the corresponding output vectors  $\vec{y}_1, \vec{y}_2, \dots, \vec{y}_t$ . The vectors all have dimension  $k$ .
2. To produce output vector  $\vec{y}_i$ , the self attention operation simply takes a **weighted average over all the input vectors**  $\vec{y}_i = \sum_j w_{ij} \vec{x}_j$ .
3. where  $j$  indexes over the whole sequence and the weights sum to one over all  $j$ . The weight  $w_{ij}$  is not a parameter, as in a normal neural net, but is *derived* from a function over  $\vec{x}_i$  and  $\vec{x}_j$ .
4. The simplest option for this function is the dot product:
  1.  $w'_{ij} = \vec{x}_i^T \vec{x}_j$ .
  2. Note:  $\vec{x}_i$  is the input vector at the same position as the current output vector  $\vec{y}_i$ . For the next output vector, we get an entirely new series of dot products, and a different weighted sum.
5. The dot product gives us a value anywhere between negative and positive infinity, so we apply a softmax to map the values to  $[0, 1]$  and ensure that they sum to 1 over the whole sequence:

$$1. w_{ij} = \frac{\exp w'_{ij}}{\sum_j \exp w'_{ij}}.$$

6. And that's the basic operation of self attention.

3. from [3], a simple description of self-attention differs with fully-connected layers:

1. Ignoring details like normalization, biases, etc, fully connected networks are fixed (in position to the features) weights:

1.  $f(x) = (Wx)$  where  $W$  is learned in training, and fixed in inference.

2. Self-attention layers are dynamic, changing the weight as it goes:

1.  $\text{attn}(x) = (Wx)$

2.  $f(x) = (\text{attn}(x) * x)$

4. How attention works, from [4]:

1. Special case of attention:

1. We have a query vector  $\vec{q}$  (e.g. the decoder state)

2. We have input vectors  $H = [\vec{h}_1, \dots, \vec{h}_L]^T$  (e.g. one per source word, encoder state)

3. We compute affinity scores  $s_1, \dots, s_L$  by "comparing"  $q$  and  $H$ . ( $H$  appeared here)

4. We convert these scores to probabilities:

1.  $\vec{p} = \text{softmax}(\vec{s})$

5. We use this to output a representation as a weighted average:

1.  $c = H^T \vec{p} = \sum_{i=1}^L p_i \vec{h}_i$  (and here)

2. Affinity scores (attention scores):

- Several ways of "comparing" a query  $\vec{q}$  and an input "key" vector  $\vec{h}_i$ :

1. Additive attention (Bahdanau et al. 2015):  $s_i = u^T \tanh(A\vec{h}_i + B\vec{q})$

2. Bilinear attention (Luong et al., 2015):  $s_i = \vec{q}^T U \vec{h}_i$

3. Dot product attention (Luong et al. 2015): particular case of bilinear attention; queries and keys must have the same size,  $s_i = \vec{q}^T \vec{h}_i$ ,  $O(n^2)$  complexity.

- The last two are easy to batch for multiple queries and multiple keys.

3. The input vectors  $H = [\vec{h}_1, \dots, \vec{h}_L]$  appear in two places:

1. They are used as keys to "compare" them with the query vector  $\vec{q}$  to obtain the affinity scores.

2. They are used as values to form the weighted average  $\vec{c} = H^T \vec{p}$ .

4. To be fully general, they don't have to be the same -- we can have:

1. A key matrix  $K = [\vec{k}_1, \dots, \vec{k}_L]^T \in \mathbb{R}^{L \times d_K}$

2. A value matrix  $V = [\vec{v}_1 \dots \vec{v}_L]^T \in \mathbb{R}^{L \times d_V}$

5. We have a more general version of attention mechanism:

1. We have a query vector  $\vec{q}$  (e.g. the decoder state)

2. We have key vectors  $K = [\vec{k}_1 \dots \vec{k}_L]^T \in \mathbb{R}^{L \times d_K}$

3. and value vectors  $V = [\vec{v}_1 \dots \vec{v}_L]^T \in \mathbb{R}^{L \times d_V}$ , (one of each per source word)

4. We compute query-key affinity scores  $s_1, \dots, s_L$  comparing  $q$  and  $K$ .

5. We convert these scores to probabilities:  $p = \text{softmax}(\vec{s})$

6. We output a weighted average of the values:  $c = V^T \vec{p} = \sum_{i=1}^L p_i \vec{v}_i \in \mathbb{R}^{d_V}$

6. Self-attention for a sequence of length  $L$ :

1. Query vectors

2. Key vectors

3. Value vectors

5. Transformers can [7]:

1. process an unordered set of tokens/vectors

1. Positional encoding adds spatial information.

2. self-attention is like clustering and replacing vectors with centers, except

1. Multiple, complex, learnable similarity functions.

2. No need to preset number of clusters.

3. Vectors are iteratively aggregated and transformed.

4. Vectors can start as purely local information (e.g. 16x16 patch).

6. What kind of word embedding is used in the original transformer?

1. Recall example of word embeddings such as one-hot, word2vec, GloVe, etc.

2. There are two options for dealing with the Pytorch nn.Embedding weight matrix: [9]
  1. Initialize it with pre-trained embeddings and keep it fixed, in which case it's really just a lookup table.
  2. Initialize it randomly, or with pre-trained embeddings, but keep it trainable. In that case the word representations will get refined and modified throughout training because the weight matrix will get refined and modified throughout training. The transformer uses a random initialization of the weight matrix and refines these weights during training - i.e. it learns its own word embeddings. Note: Recall in backpropagation we can get  $\frac{\partial L}{\partial x}$ .

#### References:

1. Formal Algorithms for Transformers, Mary Phuong, Marcus Hutter.
2. <https://peterbloem.nl/blog/transformers>
3. self attention vs fully connected layer. [stackoverflow](#).
4. Lecture 10, A. Martins, C. Zerva, M. Figueiredo, IST Lisboa.
5. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention, Angelos Karthapoulos.
6. Effective Approaches to Attention-based Neural Machine Translation. Minh-Thang Luong.
7. CS598, Derek Hoesim, UIUC.
8. Spatial Attention layer in Computer Vision: The Spatial Transformer. Youtube. Charu Aggarwal.
9. [The Transformer: Attention Is All You Need](#)